

Core Operating Systems Reference & Study Guide

Comprehensive Technical Reference: Processes, Threads, CPU Scheduling, Synchronization, Memory Management, Virtual Memory, and File Systems.

Chapter 1: Operating System Fundamentals & Kernel Architecture

An Operating System (OS) acts as an intermediary between the user and computer hardware. Its primary objectives are resource allocation, hardware abstraction, program execution, and system security.

1. Dual-Mode Operation: Modern hardware supports at least two execution modes: User Mode (Mode bit = 1) and Kernel/Supervisor Mode (Mode bit = 0). Privileged instructions (like direct I/O access, disabling interrupts, and modifying page tables) can only execute in Kernel Mode.

2. System Calls: The programmatic interface between user processes and the OS kernel. Invoked via software interrupts (traps). Examples: `fork()`, `exec()`, `wait()`, `read()`, `write()`, and `open()`.

3. Kernel Architectures:

- *Monolithic Kernel:* Entire OS runs in kernel space (e.g. Linux). High performance due to direct function calls, but failure in one module can crash the entire kernel.
- *Microkernel:* Minimalist core (IPC, basic memory, low-level scheduling) in kernel space; servers (file system, networking) run in user space (e.g. Mach, QNX). High modularity and stability, but message-passing IPC overhead.

Chapter 2: Process Management & Inter-Process Communication

A process is a program in execution. It consists of the program counter, registers, stack (temporary data/functions), heap (dynamically allocated memory), and data section (global variables).

Process Control Block (PCB): Contains Process ID (PID), Process State, Program Counter, CPU Registers, Memory Limits, and List of Open Files.

Process State Model: Five standard states: *New* (being created), *Ready* (in memory waiting for CPU), *Running* (executing instructions on CPU), *Waiting/Blocked* (waiting for I/O or event), and *Terminated*.

Context Switching: Saving the state of the currently running process (registers, PC into PCB) and loading the saved state of the next selected process. Context switch time is pure overhead because the system does no useful work during the switch.

Threads vs Processes: A thread is a basic unit of CPU utilization (lightweight process). Threads of the same process share code, data, and OS resources (files), but each thread possesses its own Thread ID, program counter, register set, and stack.

Inter-Process Communication (IPC): Shared Memory (fastest, requires synchronization) vs Message Passing (pipes, message queues, sockets, well-suited for distributed systems).

Chapter 3: CPU Scheduling Algorithms

CPU scheduling decides which ready process is allocated the CPU core by the dispatcher. Common metrics include CPU Utilization, Throughput, Turnaround Time (Completion Time - Arrival Time), Waiting Time (Turnaround Time - Burst Time), and Response Time.

Algorithm	Type	Key Advantage	Disadvantage / Hazard
First-Come, First-Served (FCFS)	Non-preemptive	Simple, fair arrival order	Convoy Effect (long processes block short ones)
Shortest Job First (SJF)	Non-preemptive	Provably minimal average waiting time	Starvation of long processes; impossible to know exact job lengths
Shortest Remaining Time First (SRTF)	Preemptive SJF	Optimal for responsive processing	High context switch overhead; starvation risk
Round Robin (RR)	Preemptive	Fair, responsive, no starvation	Performance heavily depends on Time Quantum (q); high overhead if q is small
Priority Scheduling	Preemptive / Non-preemptive	Critical jobs get immediate CPU	Indefinite blocking / Starvation (mitigated via Aging)

Chapter 4: Process Synchronization & Deadlocks

Concurrent access to shared data can lead to data inconsistency (Race Condition). The Critical Section Problem requires: 1. Mutual Exclusion, 2. Progress, 3. Bounded Waiting.

Synchronization Primitives:

- **Mutex:** A locking mechanism (binary: locked/unlocked) owned strictly by the thread that locked it.
- **Semaphore:** An integer variable accessed via wait() / P() and signal() / V(). Counting Semaphore (controls resource instances) and Binary Semaphore (acts as a mutex without ownership restrictions).
- **Classic Problems:** Producer-Consumer (bounded buffer), Readers-Writers problem, and Dining Philosophers.

Deadlocks: A situation where a set of processes are blocked because each is holding a resource and waiting for another resource acquired by another process.

Four Coffman Conditions for Deadlock:

1. *Mutual Exclusion:* Non-shareable resource.
2. *Hold and Wait:* Process holds at least one resource and waits for others.
3. *No Preemption:* Resources cannot be forcibly seized.
4. *Circular Wait:* Set {P0, P1, ..., Pn} exists such that P0 waits for P1, P1 waits for P2, and Pn waits for P0.

Banker's Algorithm: Deadlock avoidance for multiple instances of resources. Tracks Available[m], Max[n][m], Allocation[n][m], and Need[n][m] where Need = Max - Allocation. Checks if granting a request leaves the system in a Safe State where a Safe Sequence exists.

Chapter 5: Memory Management & Virtual Memory

Memory management handles physical RAM allocation to processes. Modern operating systems employ Virtual Memory to give processes the illusion of contiguous, dedicated address space exceeding physical RAM.

Paging: Physical memory divided into fixed-size frames; logical memory into fixed-size pages. Eliminates external fragmentation. Translation uses a Page Table, accelerated by Translation Lookaside Buffer (TLB).

Effective Access Time (EAT): $EAT = Hit_Ratio * (TLB_access + Memory_access) + (1 - Hit_Ratio) * (TLB_access + 2 * Memory_access)$.

Segmentation: Logical division according to programmer's view (code segment, data segment, stack). Can cause external fragmentation.

Virtual Memory & Demand Paging: Pages are brought into RAM only when referenced. If an unmapped page is accessed, the hardware generates a Page Fault Trap.

Page Replacement Algorithms:

- *FIFO*: Replaces oldest page. Suffers from Belady's Anomaly (more frames can produce more page faults).
- *Optimal (OPT)*: Replaces the page that will not be used for the longest period in the future. Theoretical benchmark.
- *Least Recently Used (LRU)*: Replaces page unreferenced for the longest time. Approximation via Clock algorithm / Second Chance.

Thrashing: When a process spends more time paging (swapping pages in and out) than executing instructions. Occurs when total size of active working sets exceeds available physical memory frames.

Chapter 6: File Systems & Storage Management

File systems provide persistent storage and directory structures (hierarchical tree).

File Allocation Methods: Contiguous Allocation (fast sequential access, external fragmentation), Linked Allocation (no external fragmentation, slow random access), and Indexed Allocation (inode table with direct, indirect, double-indirect blocks).

Disk Scheduling Algorithms: FCFS, SSTF (Shortest Seek Time First, risk of starvation), SCAN (Elevator algorithm, sweeps back and forth), C-SCAN (Circular SCAN, provides uniform wait time), and LOOK / C-LOOK.